Benchmarking Lightweight AI-Agent Frameworks on a 2 GB Raspberry Pi 4:

Framework Overhead, Offline Feasibility, and Fleet Density

SAJANA YASAS DEHIGASPITIYAGE DON
Independent Researcher
yasas@idersolutions.com

June 2026

Abstract

A new class of "lightweight" AI-agent frameworks marketed for edge deployment appeared in 2026, yet no clean, apples-to-apples measurements exist for how they actually behave on constrained hardware. We benchmark five such frameworks (ZeroClaw, PicoClaw, NanoBot, smolagents, and Agent Zero) on a single dedicated Raspberry Pi 4 with 2 GB of RAM, holding the underlying language model constant so that the differences we measure reflect *framework overhead* (memory footprint, agent-loop latency, out-of-the-box capability) rather than inference cost. Four frameworks ran and produced 100 trials across a five-task suite (five trials each); the fifth (Agent Zero) could not be installed or run on the device and is reported as a documented negative result. Three findings stand out. First, on a hosted (cloud) model all four runnable frameworks reach high task success, but their memory footprints separate cleanly by runtime: the compiled agents are 3 to 6 times lighter (ZeroClaw, Rust, ≈20 MB; PicoClaw, Go, ≈31 MB) than the interpreted ones (smolagents, Python, ≈90 MB; NanoBot, Python, ≈130 MB). Second, on a fully offline (local) model *no* agent completed even the simplest task: across eight agent×model combinations the two feasible local models fail for opposite reasons, a capable model (Qwen2.5-1.5B) being too slow for the frameworks' default network timeouts and a fast model (SmolLM2-360M) lacking the tool-calling the frameworks require. The model runs offline; the agents do not. Third, as a positive counterpoint, a single 2 GB Pi hosts eight concurrent instances of every framework at 100 % success with linear RAM growth, and the compiled frameworks pack roughly 10 to 20 times denser than the Python ones. We release the harness, raw data, and analysis to make the comparison reproducible.

Keywords: edge computing, AI agents, LLM tooling, Raspberry Pi, benchmarking, on-device inference, reproducibility.

1 Introduction

Autonomous "agent" frameworks that wrap a large language model (LLM) in a tool-using control loop have proliferated rapidly. A 2026 wave of these tools advertises itself as *lightweight* and suitable for edge or on-device deployment. The marketing claim is attractive: a credit-card-sized, roughly $35 computer running a useful autonomous agent. Whether that claim survives contact with real constrained hardware, and whether such an agent can run without any cloud dependency, has not been measured carefully.

This paper answers a single, concrete question:

> *Which lightweight (2026-era) AI-agent framework runs best on a 2 GB Raspberry Pi 4, and can any of them run fully offline?*

Contribution. Our contribution is a *reproducible* benchmark that compares modern lightweight agent frameworks on identical, constrained edge hardware while holding the language model constant. This design isolates *framework overhead*, the memory footprint, the agent-loop latency, and the out-of-the-box capability of each framework, from *inference cost*, which is identical across frameworks by construction. To our knowledge, no clean, comparable numbers for these specific tools previously existed. We contribute (i) a small, fully programmatic five-task suite with deterministic pass checks, (ii) an identical-core benchmark harness that runs every framework the same way, (iii) 100 cloud trials plus offline-feasibility probes and a concurrency-scaling study, and (iv) a documented negative result (a framework that does not qualify as lightweight on this hardware).

Scope and framing. We state three boundaries explicitly so they are not mistaken for flaws. First, we benchmark each framework *at its sensible out-of-the-box defaults*, because that is what a person deploying on a Pi actually gets; we do not claim to measure a framework "in isolation" from its own default system prompt and agent loop. Second, the primary dataset uses a hosted model over the network (the "cloud" backend); the offline ("local") backend is treated as a separate feasibility study. Third, "lightweight" is part of the inclusion criterion: a framework that cannot be installed or run headless on the device is itself a result, not missing data.

2 Related Work

Most existing evaluations of LLM agents measure *task capability* on a hosted, high-end model. AgentBench assesses LLM-as-agent reasoning and decision-making across eight interactive environments [13], and GAIA poses real-world assistant tasks that require multi-step tool use and information integration [14]. Both characterize what an agent can *do*, not what the surrounding framework *costs* to run on a given device. The control loop that the frameworks we study implement descends from ReAct, which interleaves reasoning and tool-use actions in a single model loop [12]. On the model side, a parallel line of work targets small, on-device language models, including the Qwen2.5 series [10] and SmolLM2 [11], served on edge runtimes such as llama.cpp [7] and Ollama [6]; device-level ML benchmarks in turn typically report model inference throughput rather than agent-framework overhead. What is missing, and what this paper provides, is a comparison of the agent *frameworks* themselves on fixed constrained hardware with the language model held constant, so that memory footprint, agent-loop latency, and out-of-the-box capability are isolated from inference cost. The library we adapt for one of our subjects, smolagents [5], is representative of this framework layer.

3 Experimental Design

3.1 Device under test

All measurements were taken *on the Pi*, never on the controlling laptop. (Benchmarking a $35 device by timing it on a workstation would be meaningless.) The Pi was freshly prepared and dedicated solely to this benchmark, with no competing workloads, so the idle baseline is stable and the per-agent footprints are uncontaminated. Table 1 lists the measured environment of the Raspberry Pi 4 used throughout [9].

Table 1: Device under test (all values measured on the Pi).

Property	Value
Device	Raspberry Pi 4 Model B Rev 1.5
RAM	2 GB (`MemTotal` = 1846 MB)
RAM at idle	263 MB used, 1581 MB available
Swap	1844 MB (unused during runs)
Storage	128 GB SD card (117 GB usable, 102 GB free)
CPU	4-core ARM Cortex-A72 (aarch64)
OS	Debian GNU/Linux 13 (trixie), 64-bit
Kernel	6.12.62+rpt-rpi-v8
Python	3.13.5 (PEP-668 externally managed)
Docker	not installed
Timing tool	GNU `time` at `/usr/bin/time`

The 2 GB RAM ceiling (1846 MB) is the binding constraint and the reason memory is our headline metric: the roughly 1581 MB available at idle is the budget every agent draws from. Python 3.13 is what blocks Agent Zero (Section 4), and PEP-668 is why each Python agent runs in its own virtual environment for clean isolation and reproducibility.

3.2 Language model held constant

Every framework was pointed at the same hosted model (`google/gemini-3.1-pro-preview` via OpenRouter's OpenAI-compatible API [8]) using the same API key, so cost and rate behavior are identical across frameworks. This is what makes the cross-framework comparison fair: the absolute latency numbers are model- and network-dependent, but the *differences* between frameworks are attributable to the frameworks. Because the chosen model is a preview release, a stable pinned model should be substituted for a camera-ready re-run; this does not affect the relative comparison reported here.

3.3 What we measure

Per trial we record three things, summarized in Table 2. A single `/usr/bin/time -f "%e %M"` invocation captures both wall-clock seconds and peak resident set size of the agent's direct child process. We confirmed that each agent does its work in one process (no forked long-running daemon), so peak RSS is a fair, comparable footprint.

Table 2: Per-trial metrics and their interpretation.

Metric	How captured	Interpretation
success (pass/-fail)	programmatic check on the agent's output	did the agent actually accomplish the task
seconds	`/usr/bin/time %e` (wall clock)	end-to-end latency including model thinking and network
peak_mem_mb	`/usr/bin/time %M` (max RSS) ÷ 1024	the framework's memory footprint

Methodological rules. We adopt small-sample-robust reporting and explicit fairness controls.

- **n = 5 trials** per task per agent. We report the *median* and the min–max range, not the mean, because the sample is small and latency has outliers.

- **Memory is the trustworthy axis; time is the noisy axis.** Cloud network adds seconds and occasional large outliers, so we trust memory differences and caveat time differences.
- **Independent trials.** State is reset between trials (files the agent created are deleted) so no trial can "pass" on a leftover artifact. The memory task is the deliberate exception (Section 3.4).
- **Session-persistence fairness control.** Frameworks differ in whether a single non-interactive invocation carries conversation context across calls. ZeroClaw and smolagents are naturally context-free per call; PicoClaw and NanoBot persist by default, so their session store is cleared before each call. Every agent therefore runs each trial context-free.
- **Out-of-the-box defaults.** No tuning of prompts or agent loops. The only per-agent configuration is what is required to point the framework at the model (plus, where a framework ships nothing usable, a documented minimal adaptation; see Section 4).

3.4 The task suite

The suite (Table 3) climbs a difficulty gradient from a single tool call to cross-session memory. All checks are deterministic. Tasks T1 to T4 write to a file that is checked on disk, which avoids fragile reply-text parsing; T5 is the exception, since recall naturally returns as a chat reply that we grep.

Table 3: The five-task suite. All pass checks are programmatic.

ID	Tests	Prompt (verbatim, abridged)	Pass check
T1	basic tool call	create `result.txt` containing exactly `BENCHMARK_OK`	file contains `BENCHMARK_OK`
T2	chaining + state	read number in `input.txt` (=14), multiply by 3, write to `output.txt`	file contains `42`
T3	using tool output	count lines in `data.txt` (7 lines), write the number to `count.txt`	file contains `7`
T4	robustness	append a line with the word `done` to `log.txt` (absent)	file contains `done`
T5	persistent memory	store "project ID is ZX-9", then in a fresh session recall it	reply contains `ZX-9`

T5 is the most discriminating task: it is executed as two separate invocations (store, then recall) with no reset between them, and for agents that persist session context the session is cleared before *both* calls so that recall can only succeed through the framework's real memory mechanism, not residual chat history. We time and check the recall call.

3.5 The benchmark harness

The harness is a set of small shell scripts, one pair per agent (one for tasks T1 to T4, one for the memory task T5). The scripts are identical in task prompts and pass checks; only the agent name, the run command, the workspace path, and the session-reset step differ. This identical-core design is what makes the comparison fair. Figure 1 shows the lifecycle of a single trial. Listing 1 shows the core of the canonical script.

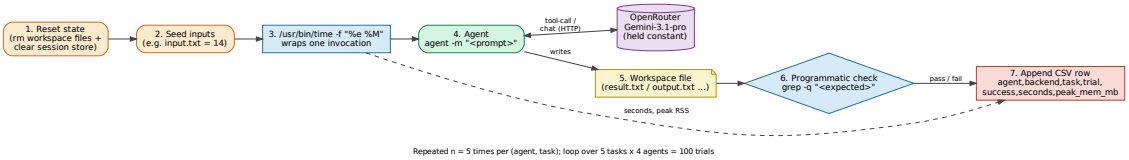

Figure 1: Lifecycle of a single benchmark trial. State is reset and inputs are seeded, then one timed agent invocation runs against the constant cloud model; the resulting workspace file is checked programmatically and one row is appended to the CSV. The loop runs $n = 5$ times per (agent, task), over 5 tasks and 4 agents, for 100 cloud trials.

```
run_agent () {                  # 1) clear session: this call has no prior context
  rm -f "$SESSIONS"/*.jsonl 2>/dev/null
  /usr/bin/time -f "%e %M" -o results/_time.txt \
    picoclaw --no-color agent -m "$1" >"$2" 2>>results/_agent_err.log
}
# task2 (chaining + state):
rm -f "$WORKSPACE/output.txt"; echo "14" > "$WORKSPACE/input.txt"   # seed
run_agent "Read the number in input.txt, multiply by 3, write it to output.txt" "$OUT"
grep -q "42" "$WORKSPACE/output.txt" && OK=1 || OK=0                # deterministic check
read SECS KB < results/_time.txt
MB=$(awk -v k="$KB" 'BEGIN{printf "%.1f", k/1024}')
echo "$AGENT,cloud,task2,$i,$([ $OK = 1 ] && echo pass || echo fail),$SECS,$MB,auto" >> "$CSV"
```

Listing 1: Core of the canonical task harness (PicoClaw shown; other agents differ only in the run command, workspace, and session-reset line).

4 The Frameworks

All four runnable frameworks were pointed at the same model with the same key. Memory footprints are flat across tasks for every agent (no per-task leak), so a single footprint number per framework is meaningful. Table 4 summarizes the lineup.

Table 4: The frameworks under test. Footprint is the median peak RSS (flat across tasks).

Framework	Lang.	Version	Footprint	Memory mechanism
ZeroClaw	Rust	0.7.3	≈20 MB	dedicated store, synchronous recall
PicoClaw	Go	0.2.9	≈31 MB	dedicated store, synchronous recall
smolagents	Python	1.26.0	≈90 MB	none built in (improvised)
NanoBot	Python	0.2.1	≈130 MB	asynchronous "dream" consolidation
Agent Zero	Python	—	not runnable	excluded with cause

ZeroClaw (Rust). A compiled native binary [1]; the lightest of all. Out of the box it is *secure by default*: in non-interactive mode it refuses file writes because no human is present

to approve the action. Raising autonomy to full resolves this. It is the only framework that required loosening to perform the tasks, and that default is itself a finding.

PicoClaw (Go). A single static binary [2]. Writes files freely out of the box (the opposite default to ZeroClaw). It persists session context by default, so we clear its session store before each call.

NanoBot (Python). The heaviest runnable framework [3]. Writes files freely. Its long-term memory is consolidated *asynchronously* by a background process, which becomes the mechanism behind its only failure (Section 5.5).

smolagents (Python library). Not a turnkey application but a library [5] shipping neither a CLI nor file tools. To benchmark it on the same tasks we wrote a minimal runner that builds the flagship code-writing agent over the same OpenAI-compatible endpoint, equipped with three minimal file tools. This is a documented adaptation and is itself a finding: smolagents requires assembly that the turnkey agents do not.

Agent Zero (excluded with cause). Agent Zero [4] could not be installed or run on the device: its requirements do not install on the Pi's Python 3.13 (a dependency pins Python below 3.13), it offers no headless or CLI interface (web-UI / Docker only, so it is not measurable per-invocation), and it carries a heavyweight Docker-oriented dependency stack. It is not a lightweight edge agent. This is a result, not missing data.

5 Cloud Results

This section uses the primary dataset of 100 trials (4 agents × 5 tasks × 5 trials) on the hosted model.

5.1 Success

All four runnable frameworks pass T1 to T4 at 100 %. The single exception is NanoBot on the memory task (T5), which fails all five trials. Figure 2 shows the full success matrix; the mechanism behind the NanoBot failure is analyzed in Section 5.5.

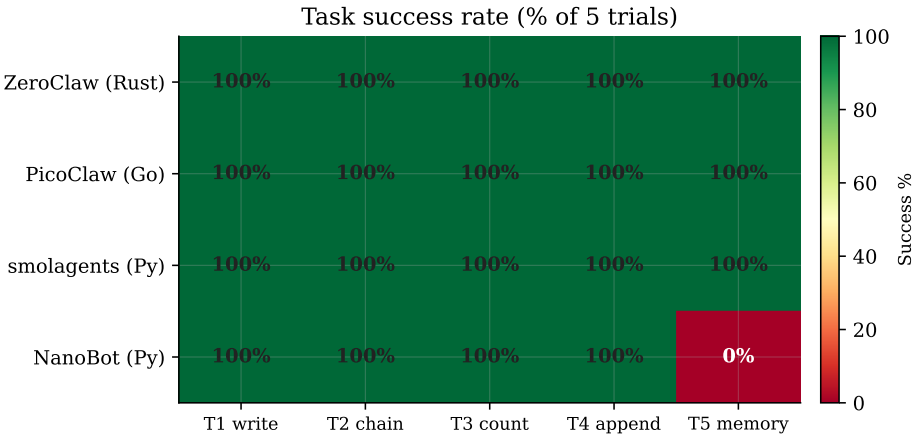

Figure 2: Task success rate (percentage of five trials). Three frameworks are perfect; NanoBot fails only the persistent-memory task.

5.2 Memory footprint, the clean separator

Peak memory is flat across tasks for every framework, and the per-framework values separate cleanly by runtime (Figure 3). The two compiled-language agents are 3 to 6 times lighter than the Python ones. Because memory is the trustworthy axis (no network noise), this ordering is the most robust result in the paper.

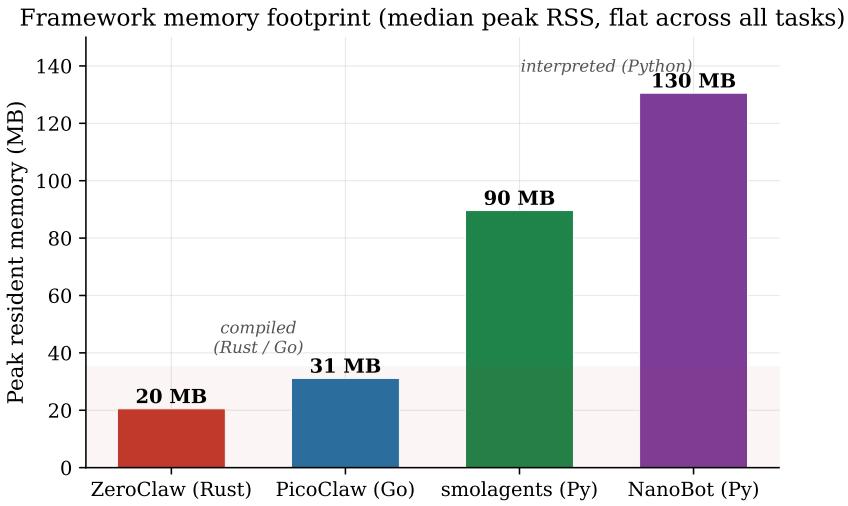

Figure 3: Median peak resident memory per framework. The ordering tracks the runtime exactly: Rust $<$ Go $\ll$ Python.

5.3 Latency

Latency tracks the number of model round-trips and carries high network-driven variance, so we read it cautiously (Figure 4). PicoClaw is fastest overall; smolagents' code-writing loop has the worst tail (a single 143 s run on T2, off the chart). NanoBot is consistently the slowest of the four. We emphasize that single-run latency differences within a few seconds are within network noise; only large and consistent gaps should be read as real.

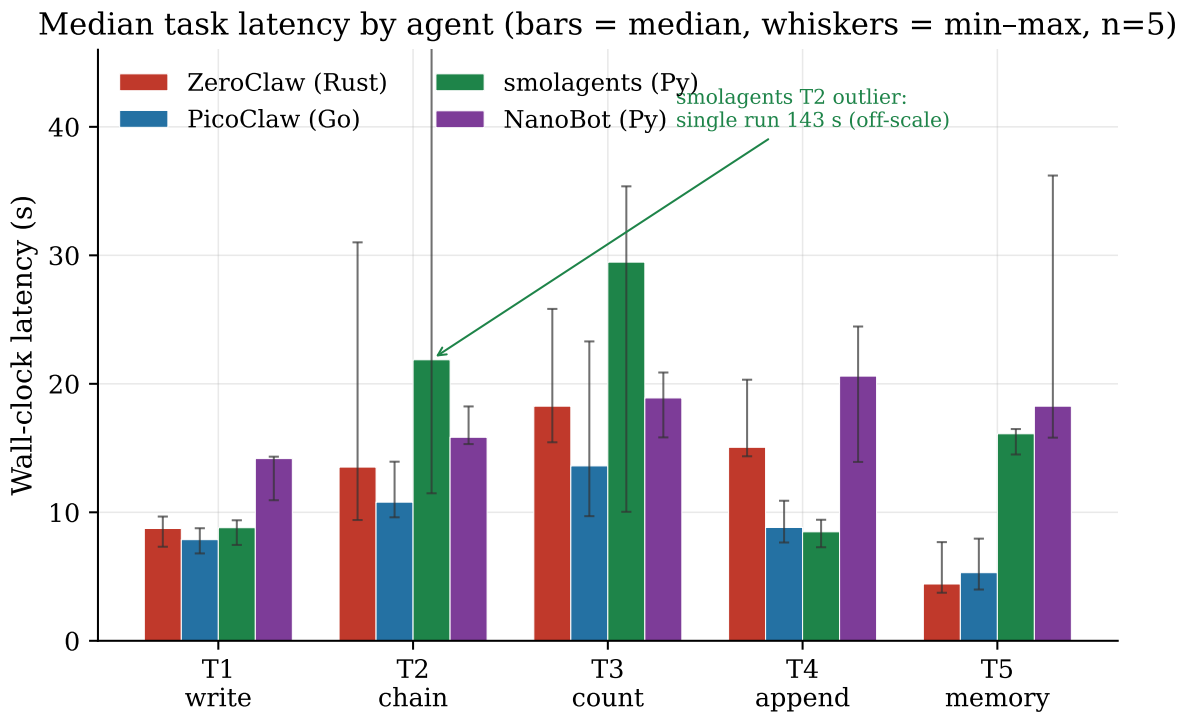

Figure 4: Median task latency by framework (bars = median, whiskers = min–max, $n = 5$). The smolagents T2 distribution includes a 143 s outlier shown off-scale.

5.4 Cross-framework summary

Table 5 consolidates the comparison.

Table 5: Cross-framework cloud results ($n = 5$, median seconds; footprint in MB; success out of 25).

	ZeroClaw	**PicoClaw**	**smolagents**	**NanoBot**
Median latency (s)				
T1 write	8.74	7.87	8.80	14.18
T2 chain	13.52	10.79	21.87	15.84
T3 count	18.26	13.61	29.47	18.90
T4 append	15.06	8.82	8.48	20.60
T5 memory	4.42	5.30	16.11	18.26
Footprint (MB)	≈20.5	≈31.1	≈89.6	≈130.5
Success	25/25	25/25	25/25	**20/25**

5.5 Why the memory task splits frameworks

T5 is the only task that separates the runnable frameworks, and it does so along an architectural line rather than a performance one (Figure 5). ZeroClaw and PicoClaw write to a dedicated memory store *synchronously* at store time, so a fresh-session recall succeeds. smolagents has no memory feature at all, yet passes by improvising filesystem-as-memory: its code-writing agent writes a file during the store call and rediscovers it by listing the directory during recall, a different mechanism reaching the same outcome. NanoBot consolidates long-term memory *asynchronously* through a background process that has not run by the time recall is issued, so with the session wiped it cannot retrieve the fact. This is a genuine capability differentiator, not

a tuning artifact.

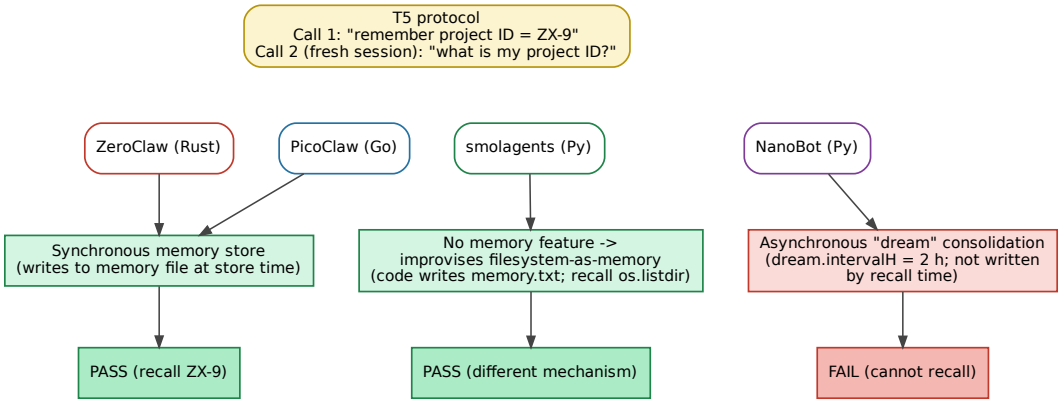

Figure 5: Why the persistent-memory task (T5) splits the frameworks. The outcome depends on *when* each framework commits a remembered fact to durable storage relative to the recall call.

5.6 Reproducibility

ZeroClaw was independently re-run on a separate occasion. Both runs pass 25/25, and the per-task medians track closely (Figure 6), supporting the stability of the harness and the constant-model design.

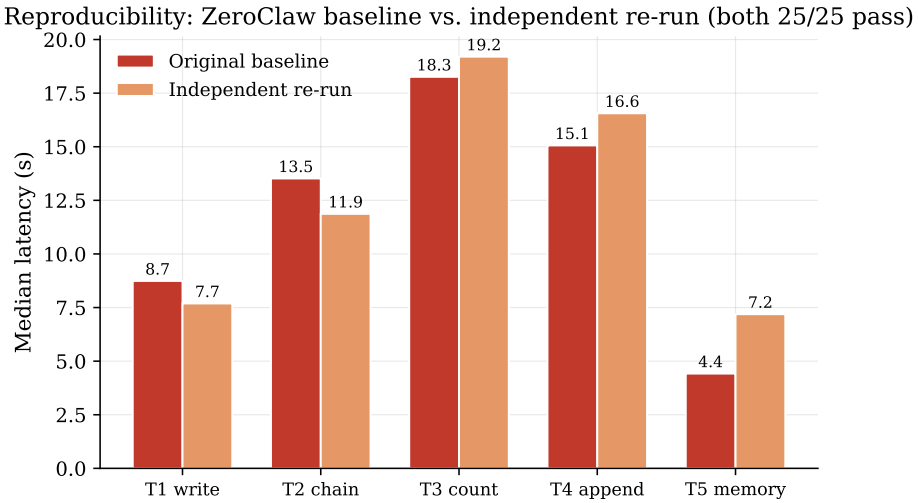

Figure 6: Reproducibility check: ZeroClaw original baseline versus an independent re-run. Both pass every trial and the medians agree within network variance.

6 Offline Feasibility

We next asked whether any framework can run *fully offline*, swapping the hosted model for a local one served on the Pi's CPU. The runtime was Ollama [6] (built on the llama.cpp inference engine [7]); the two locally-runnable models that bracket the feasible size band on 2 GB are

Qwen2.5-1.5B [10] (986 MB, capable, with tool-calling) and SmolLM2-360M [11] (725 MB, about five times faster but weaker and without tool-calling). We ran one T1 probe per framework against each model.

Result: 0/8. No agent completed even T1 with either local model, and the two models fail for opposite reasons (Figure 7, Table 6). The capable model is too slow: at roughly 3.3 tokens per second its time-to-first-token on a 5 to 10k-token agent prompt exceeds the frameworks' default HTTP timeouts, so ZeroClaw and PicoClaw abort before the model answers, and the code-writing loop of smolagents never converges within any reasonable cap. The fast model is too weak and exposes no tool-calling, so the tool-using frameworks (PicoClaw, NanoBot) are rejected with a hard HTTP 400 in seconds, while smolagents survives that (it drives the model with code, not tool calls) but the 360M model emits malformed code.

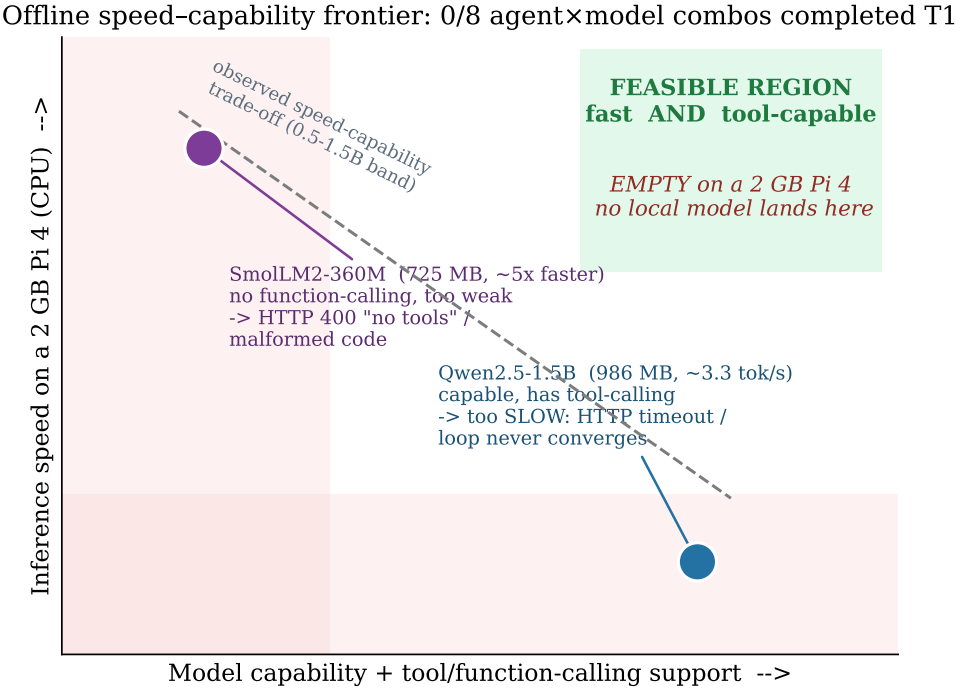

Figure 7: The offline speed–capability frontier on a 2 GB Pi 4. The two feasible local models occupy opposite corners; the region that is both fast enough and capable enough to drive these frameworks is empty on this hardware.

Table 6: Offline frontier: outcome of one T1 probe per (agent, model). No combination completed the task.

Framework	Qwen2.5-1.5B (capable, slow)	SmolLM2-360M (fast, no tools)
smolagents	timeout >1200 s	timeout >1200 s (malformed code)
PicoClaw	HTTP timeout (≈6 min)	HTTP 400 no-tools (1.25 s)
NanoBot	slow / over cap	HTTP 400 no-tools (8.27 s)
ZeroClaw	HTTP timeout (≈5 min)	fail at 807 s

The publishable core. The bottleneck on a 2 GB Pi 4 is *speed and capability, not RAM*: the capable model fits in memory (about 1.4 GB at a 10k-token context, leaving roughly 288 MB free) but is about six times too slow for an interactive loop, while the fast model lacks the tool interface the frameworks need. The two failure modes are a clean, specific finding: the

frameworks are simply not tuned for inference this slow, and a timeout-versus-slow-inference mismatch makes them abort before the model can answer. The model runs offline; the agents do not.

7 Concurrency and Fleet Density

A different and more encouraging axis: how many instances of a framework can one 2 GB Pi run at once? We launched $N = 1, 2, 4, 8$ concurrent same-agent instances on T1 against the cloud model, each with a unique session and output file, and measured per-instance latency, peak combined system RAM, and success. (Local is excluded here because the local server processes one request at a time, so "concurrency" there merely queues.)

Success: 100 % everywhere. Every framework passed every instance at every N (8/8 even at $N = 8$). On the cloud backend a 2 GB Pi comfortably hosts a fleet of these agents concurrently.

RAM grows linearly. Combined system RAM grows linearly with N, and the per-instance cost is essentially the single-agent footprint (Figure 8). Even eight copies of the heaviest framework (NanoBot at 1166 MB peak) leave several hundred MB free.

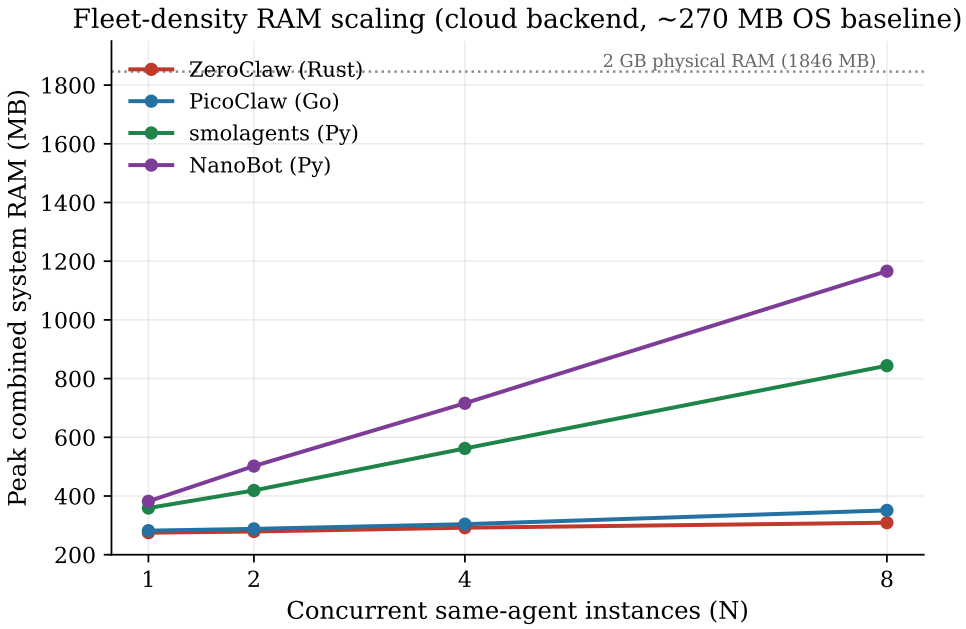

Figure 8: Peak combined system RAM versus the number of concurrent instances. Growth is linear; the compiled frameworks stay near the OS baseline.

Latency under load. The compiled frameworks stay flat under concurrency because they are network-I/O bound and eight concurrent calls overlap cleanly. The Python frameworks show modest latency growth at $N \geq 4$ from interpreter CPU contention on the four-core Pi, not failures, just slowdown (Figure 9).

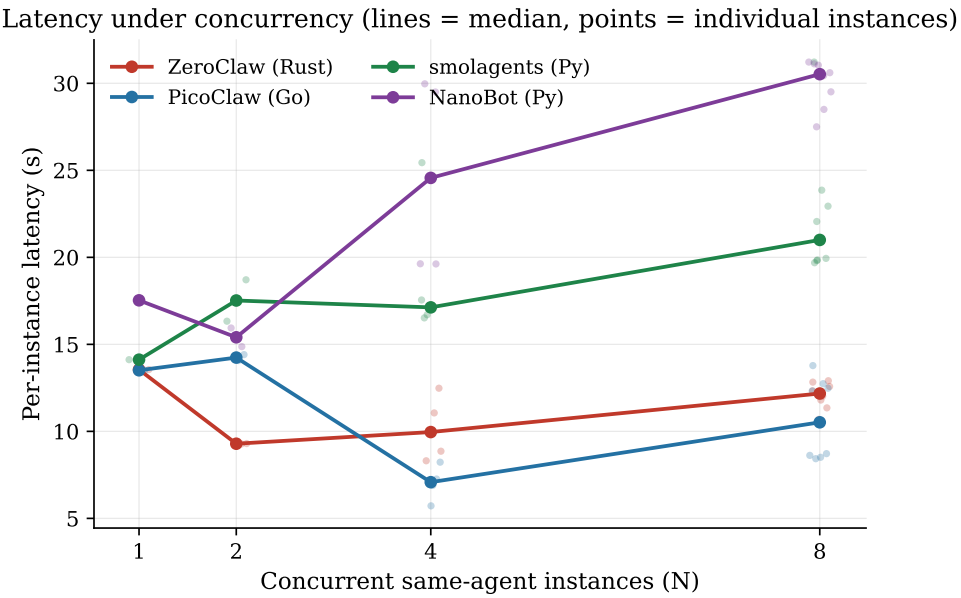

Figure 9: Per-instance latency versus concurrency (lines = median, points = individual instances). Compiled frameworks are flat; Python frameworks grow mildly.

Density ceiling. Extrapolating from the per-instance footprint against the roughly 1.55 GB usable for agents gives the RAM-bound ceilings in Figure 10. The compiled frameworks pack roughly 10 to 20 times denser than the Python ones; in practice their limiter is the network or API rate, not the Pi's RAM. Table 7 gives the measured scaling numbers.

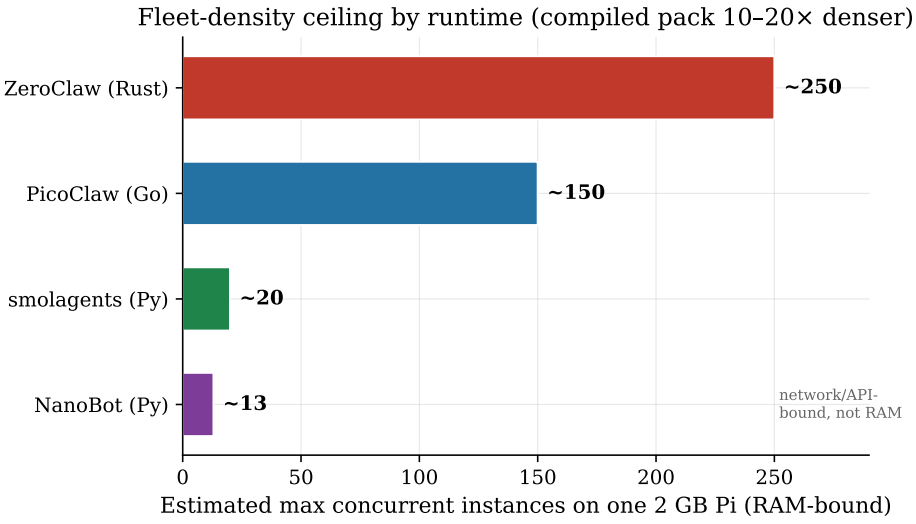

Figure 10: Estimated maximum concurrent instances on one 2 GB Pi (RAM-bound). Confirmed empirically to $N = 8$; the compiled frameworks are network-bound well before they are RAM-bound.

Table 7: Measured concurrency scaling. RAM is peak combined system RAM (MB) over a ≈270 MB OS baseline; latency is median per-instance (s).

Framework	N=1	N=2	N=4	N=8	≈/instance
Peak combined RAM (MB)					
ZeroClaw	275	279	292	309	≈5 MB
PicoClaw	282	288	304	351	≈10 MB
smolagents	359	419	562	844	≈72 MB
NanoBot	382	502	716	1166	≈112 MB
Median per-instance latency (s)					
ZeroClaw	13.6	9.3	10.0	12.2	flat
PicoClaw	13.5	14.2	7.1	10.5	flat
smolagents	14.1	17.5	17.1	21.0	mild ↑
NanoBot	17.5	15.4	24.6	30.5	↑

8 Threats to Validity

Construct validity. We measure frameworks at their out-of-the-box defaults, so a framework's default prompt and agent loop are part of what we measure. We state this rather than claim to isolate the framework from its own behavior; for the deployment question we pose, the default is the right unit of analysis.

Internal validity. Latency carries network variance, which is why we report medians over five trials with min–max ranges and treat memory as the headline. The constant model, constant key, and identical harness core hold the major confounds fixed. The independent ZeroClaw re-run (Figure 6) supports stability.

External validity. The cloud results depend on a specific (and preview) hosted model and on network conditions, so absolute latencies will not transfer; the relative framework ordering and the memory footprints should. The offline study uses two models that bracket the feasible size band on 2 GB; we deliberately skipped a third model of the same size and lower capability because it would only reproduce one endpoint's failure with no new information.

Statistical scope. With $n = 5$ we make ordinal claims (which framework is lighter, which fails which task) rather than precise effect-size estimates, which is appropriate for a systems benchmark on noisy hardware.

9 Discussion and Recommendations

Three practical conclusions follow. First, for a single-agent deployment on a constrained Pi with a hosted model, all four runnable frameworks are viable on capability, so the choice reduces to footprint and memory semantics: the compiled frameworks are markedly lighter, and if cross-session memory matters, a framework with a synchronous store is the safe default (an asynchronous consolidation design can silently fail a fresh-session recall). Second, for multi-agent or fleet deployment with a hosted model, the compiled frameworks are dramatically more economical, packing roughly an order of magnitude more instances into the same 2 GB. Third, truly offline autonomous agency is not yet practical on a 2 GB Pi 4: the constraint is not memory but the absence of a local model that is simultaneously fast enough and tool-capable enough, compounded by framework HTTP timeouts that are not tuned for slow local inference. A near-term path to offline operation would combine a small tool-capable model with framework timeouts raised to match local inference speed, or hardware with more memory bandwidth than the Pi 4.

10 Conclusion

We presented a reproducible, constant-model benchmark of lightweight 2026-era AI-agent frameworks on a 2 GB Raspberry Pi 4. On a hosted model all runnable frameworks reach high task success, but memory footprint separates them cleanly by runtime, with the compiled (Rust, Go) frameworks 3 to 6 times lighter than the Python ones, and the persistent-memory task exposes a real architectural difference in how each framework commits remembered facts. Fully offline, no framework completed even the simplest task across eight agent-by-model combinations, failing for opposite reasons that together bound a speed-versus-capability frontier with an empty feasible region on this hardware. As a positive counterpoint, one 2 GB Pi hosts eight concurrent instances of every framework at full success with linear RAM, and the compiled frameworks pack 10 to 20 times denser. The model runs offline; the agents do not, yet, and on the cloud the lightweight compiled frameworks are the clear choice for dense edge deployment.

Future work. Substitute a stable pinned model for the preview model; add on-device temperature logging to flag thermal throttling during any longer local runs; extend the offline study with raised framework timeouts and small tool-capable models; and broaden the framework set as the ecosystem matures.

Data and Code Availability

The benchmark harness, all raw per-trial CSV data (cloud, offline, and concurrency tracks), and the analysis scripts that produce every figure and table in this paper are released to support replication at `https://github.com/ddsyasas/lightweight-agent-benchmark`. Each CSV maps to a specific experiment and section as documented in the repository README.

References

[1] ZeroClaw, a lightweight Rust agent framework. Source repository: `https://github.com/zeroclaw-labs/zeroclaw`.

[2] PicoClaw, a lightweight Go agent framework. Source repository: `https://github.com/sipeed/picoclaw`.

[3] NanoBot, a lightweight Python agent framework. Source repository: `https://github.com/HKUDS/nanobot`.

[4] Agent Zero, a general-purpose personal agent framework. Source repository: `https://github.com/agent0ai/agent-zero`.

[5] smolagents, a barebones library for building agents. Hugging Face. Source repository: `https://github.com/huggingface/smolagents`.

[6] Ollama: a local large language model runtime. `https://ollama.com`.

[7] G. Gerganov and contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`.

[8] OpenRouter: a unified API gateway for hosted language models. `https://openrouter.ai`.

[9] Raspberry Pi Foundation. Raspberry Pi 4 Model B product documentation. `https://www.raspberrypi.com`.

[10] A. Yang et al. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115, 2024.

[11] L. Ben Allal et al. SmolLM2: When Smol Goes Big, Data-Centric Training of a Small Language Model. arXiv preprint arXiv:2502.02737, 2025.

[12] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proc. ICLR*, 2023. arXiv:2210.03629.

[13] X. Liu et al. AgentBench: Evaluating LLMs as Agents. In *Proc. ICLR*, 2024. arXiv:2308.03688.

[14] G. Mialon et al. GAIA: a Benchmark for General AI Assistants. arXiv preprint arXiv:2311.12983, 2023.